

Embedded Programming

Introduction to course

Dr. Charis Kouzinopoulos

Department of Advanced Computing Sciences

Maastricht University

2024-2025

charis.kouzinopoulos@maastrichtuniversity.nl



Who am I?



Charis Kouzinopoulos

Assistant Professor of IoT

Department of Advanced Computing Sciences

charis.kouzinopoulos@maastrichtuniversity.nl

Paul-Henri Spaaklaan 1 (room C4.011)

Research interests

- *Low-power embedded systems*
- *Hardware/software co-design*
- *Training/inference of ML at the edge*

Courses

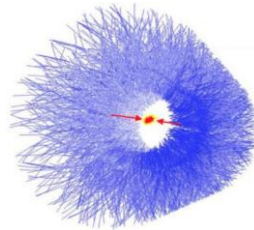
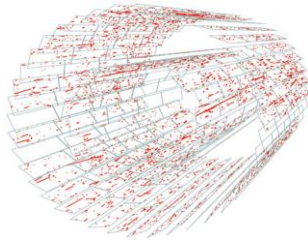
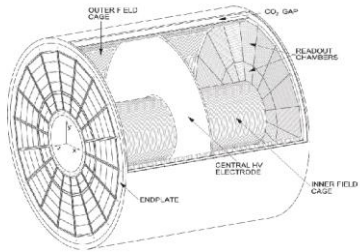
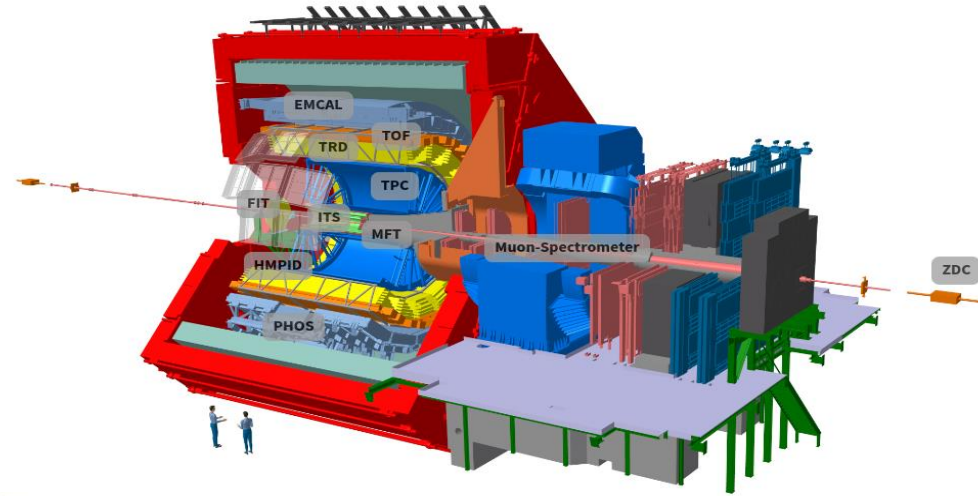
- *Procedural Programming*
- *Embedded Programming*
- *High Performance Computing*

Research background

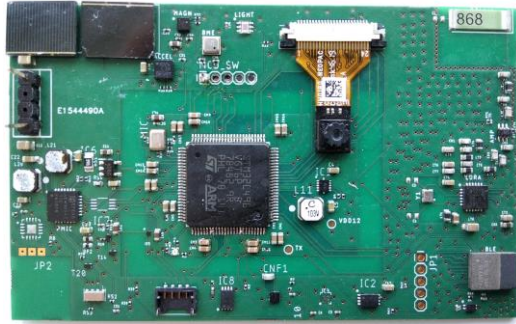
Hardware/software optimization

HPC

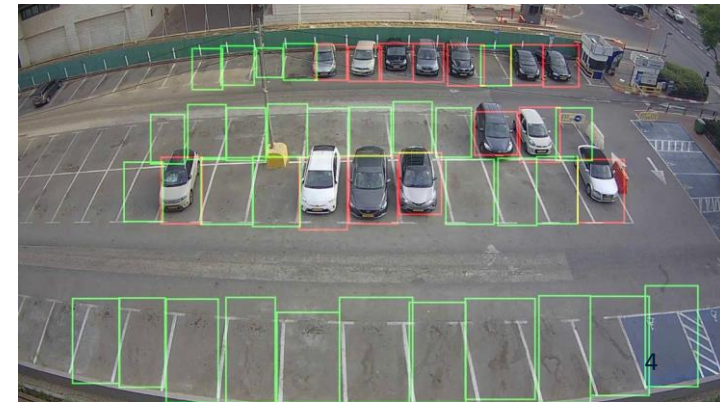
CERN



Research background



Hardware/software co-design
CERTH-ITI



But what is an embedded system?

??

But what is an embedded system?

*“a combination of a computer processor, computer memory, and input/output peripheral devices—that has a **dedicated function** within a larger mechanical or electronic system”*

But what is an embedded system?

*“a combination of a computer processor, computer memory, and input/output peripheral devices—that has a **dedicated function** within a larger mechanical or electronic system”*

- Computer system hidden (embedded) in other systems
 - Often interacts with the physical environment (CPS)
- Special-purpose vs. general-purpose computing
- End users see “smart” devices rather than computers

Embedded Systems



Amazon Warehouse





Course outline – Embedded Programming

- Primer on computer architecture
- The STM32U5 and ARM Cortex-M33 architectures
- Data representation and bitwise operations
- Pointers and pointer arithmetic using C
- ARM ISA and simple assembly language
- Fun with GPIOs and clocks

Learning objectives

- Demonstrate an understanding of **computer architecture**, with an emphasis on the ARM Cortex-M microprocessor
- Demonstrate an understanding of **signals, buses, memory hierarchies and datapaths**
- Learn a subset of the ARM 32-bit **ISA** (thumb/ARMv8-M):
 - Arithmetic and logic instructions
 - Data movement instructions
 - Procedures
 - Branching, looping and testing instructions
- Gain an in-depth understanding of **datatypes** and **number systems**
- Understand **pointers** and pointer arithmetic
- Combine all previous knowledge to interact with the **B-U585I-IOT02A discovery board**

Embedded programming - Lecture and lab sessions

- **Classic lectures, practical live-coding and tutorial sessions**
- Always bring a laptop to connect to the boards
- $4 \text{ ECTS} \times 28\text{h} = 112\text{h} \rightarrow$ on average 18h per week
- Only $6 \times 6 + 4 = 40\text{h}$ “contact”
 - Remaining 72h $\rightarrow$ self-studies, exam preparation, prepare tutorials,
 - It is estimated that you spend on average 8h-10h (max!!!) per week with self-studies

Assessment

Final exam taken using Testvision. Course assignments or any exercises such as quizzes do not count for the final grade. **Bonus** test during 4th week's tutorial for **up to 1 additional grade**

The structure of the exam can be estimated as follows:

- Theoretical questions + small coding exercises: ~40%
- Practical exercises (usually, 2-3 exercises): ~60%

The final grade is computed as follows:

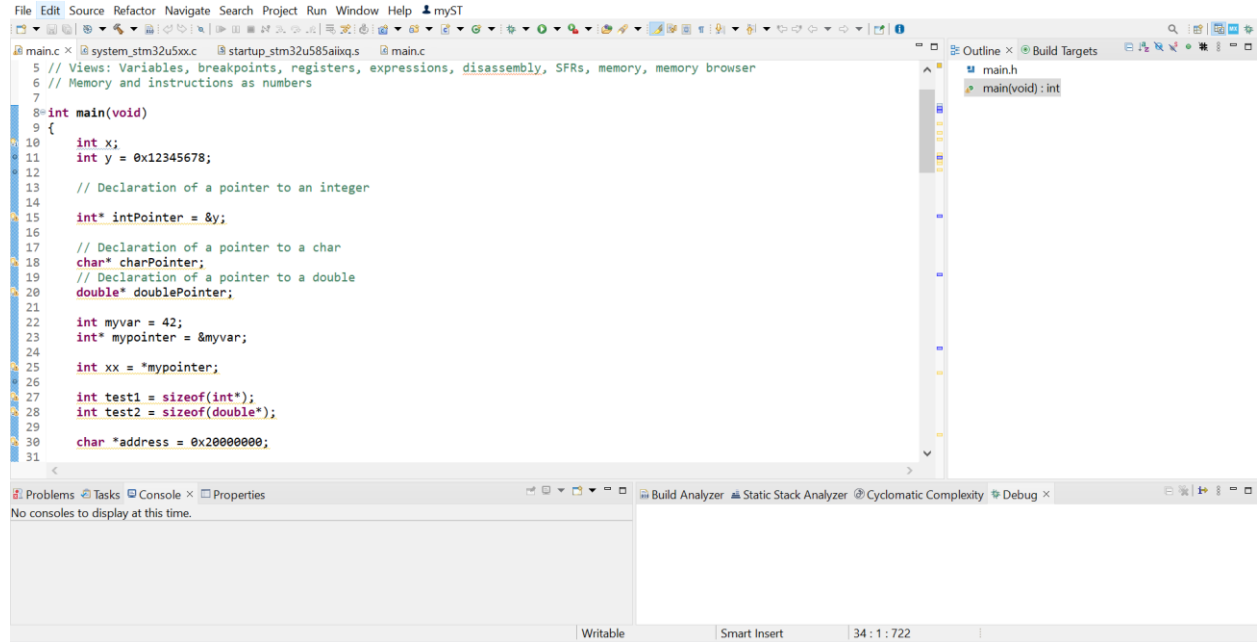
$$\text{final grade (out of 10)} = \text{exam grade} + \text{bonus grade}$$

In the case of no passing grade during the normal period, you can take a resit exam (same format as the normal period exam). In this case, your final grade will be :

$$\text{resit final grade (out of 10)} = \text{resit exam} + \text{bonus grade}$$

Software environment

- The official IDE for this course is CubeMXIDE
- Check instructions at Assignment 0



The screenshot displays the CubeMX IDE interface. The main editor window shows a C program in `main.c`. The code includes comments and declarations for variables and pointers. The `main` function is defined and contains several assignments and declarations. The IDE's interface includes a menu bar at the top, a toolbar, and a sidebar on the right with an 'Outline' view showing the project structure. At the bottom, there is a 'Problems' panel and a 'Console' panel, both currently empty.

```
File Edit Source Refactor Navigate Search Project Run Window Help myST
main.c x system_stm32u5xxx startup_stm32u585aixq.s main.c
5 // Views: Variables, breakpoints, registers, expressions, disassembly, SFRs, memory, memory browser
6 // Memory and instructions as numbers
7
8 int main(void)
9 {
10     int x;
11     int y = 0x12345678;
12
13     // Declaration of a pointer to an integer
14
15     int* intPointer = &y;
16
17     // Declaration of a pointer to a char
18     char* charPointer;
19     // Declaration of a pointer to a double
20     double* doublePointer;
21
22     int myvar = 42;
23     int* mypointer = &myvar;
24
25     int xx = *mypointer;
26
27     int test1 = sizeof(int*);
28     int test2 = sizeof(double*);
29
30     char *address = 0x20000000;
31 }
```


Reading material

- **David A. Patterson, John L. Hennessy** - Computer Organization and Design ARM Edition: The Hardware Software Interface
- **Y. Zhu** - Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C (Fourth Edition)
- **D. Harris, S. Harris** - Digital Design and Computer Architecture

Different datasheets for reference during the lectures and tutorials:

- STM32U585AI datasheet
- PM0264 - STM32 Cortex-M33 MCUs programming manual
- Arm Cortex-M33 Processor Technical Reference Manual
- Armv8-M Architecture Reference Manual
- UM2839 Discovery kit for IoT node with STM32U5 series

Assembly Programming



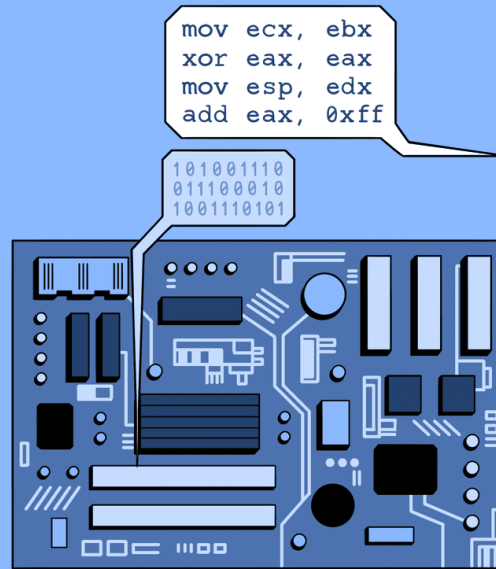
<http://www.andysinger.com/>

Embedded Systems

Performance, performance, performance!

Cost, cost, cost!

Assembly Programming



Assembly Language

[ə-'sem-blē 'laŋ-gwij]

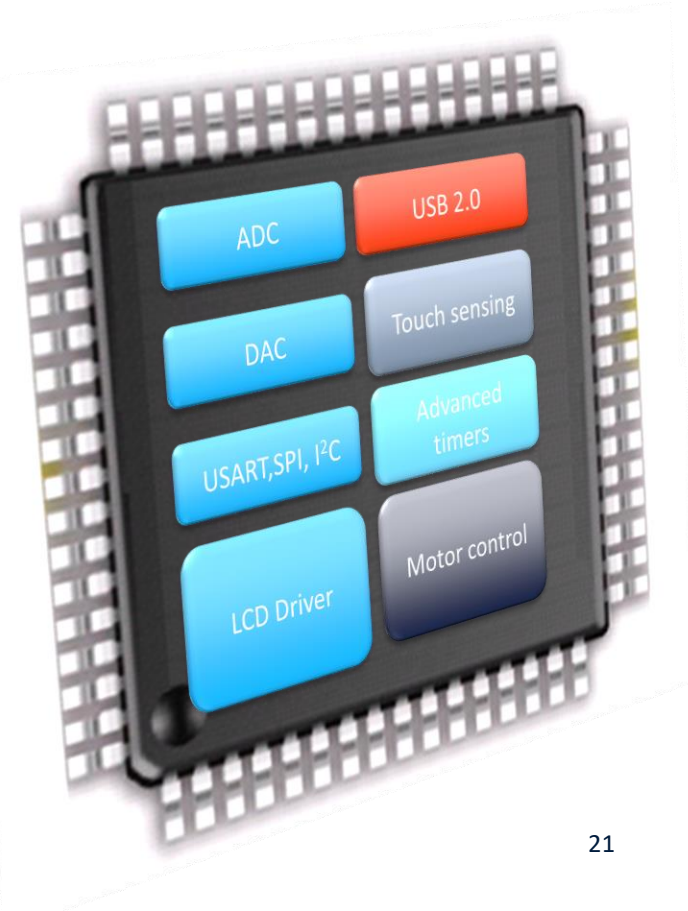
Low-level programming language intended to communicate directly with a computer's hardware.

Why should we learn Assembly?

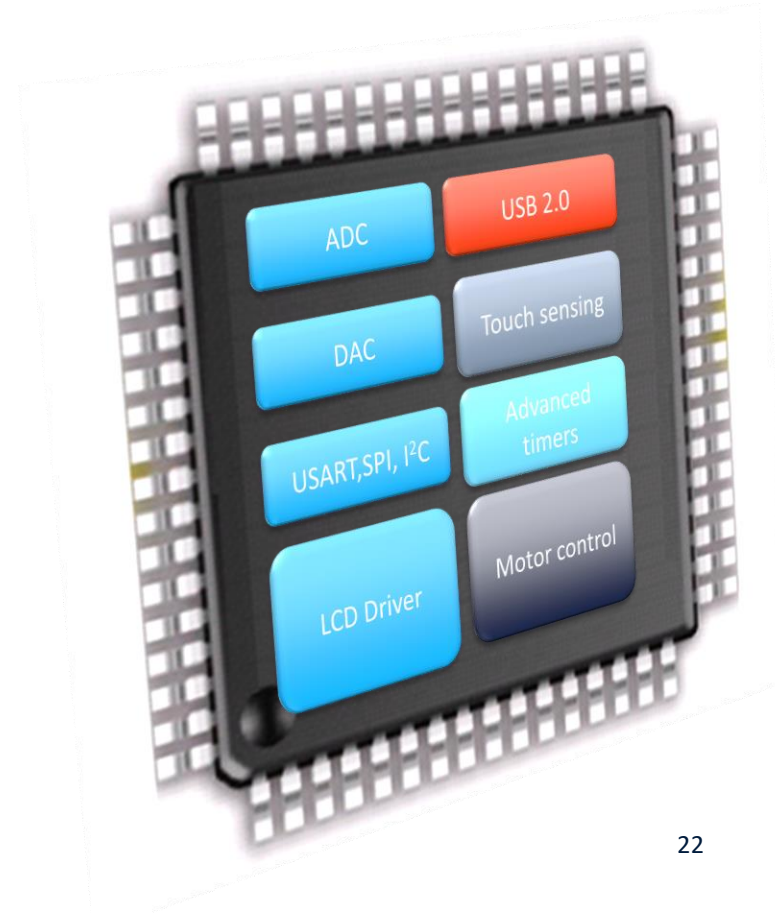
- Assembly isn't "just another language".
 - Help you understand how does the processor work
- **Assembly program runs faster than high-level language.**
 - Performance critical codes may need to be written in assembly.
 - Use the profiling tools to find the performance bottle and rewrite that code section in assembly
 - Latency-sensitive applications, such as aircraft controller
 - Standard C compilers do not use some operations available on ARM processors, such ROR (Rotate Right) and RRX (Rotate Right Extended).
- **Hardware/processor specific code,**
 - Processor booting code
 - Device drivers
 - Compiler, assembler, linker
 - A test-and-set atomic assembly instruction can be used to implement locks and semaphores.
- **Cost-sensitive applications**
 - Embedded devices, where the size of code is limited, wash machine controller, automobile controllers
- **Better understand high-level programming languages**

Why ARM processors

- ARM: **A**cron **R**ISC **M**achine, founded in 1990
- Public company, Headquarter at Cambridge, England, UK, 2023 Revenue: US\$2.68 billion
- Arm processors are used as the main CPU for most mobile phones and handhelds.
- In late 2020, Apple began the transition from Intel processors to Apple silicon in Mac computers.
- The world's second fastest supercomputer (previously fastest) in 2022, the Japanese Fugaku is based on Arm AArch64 architecture

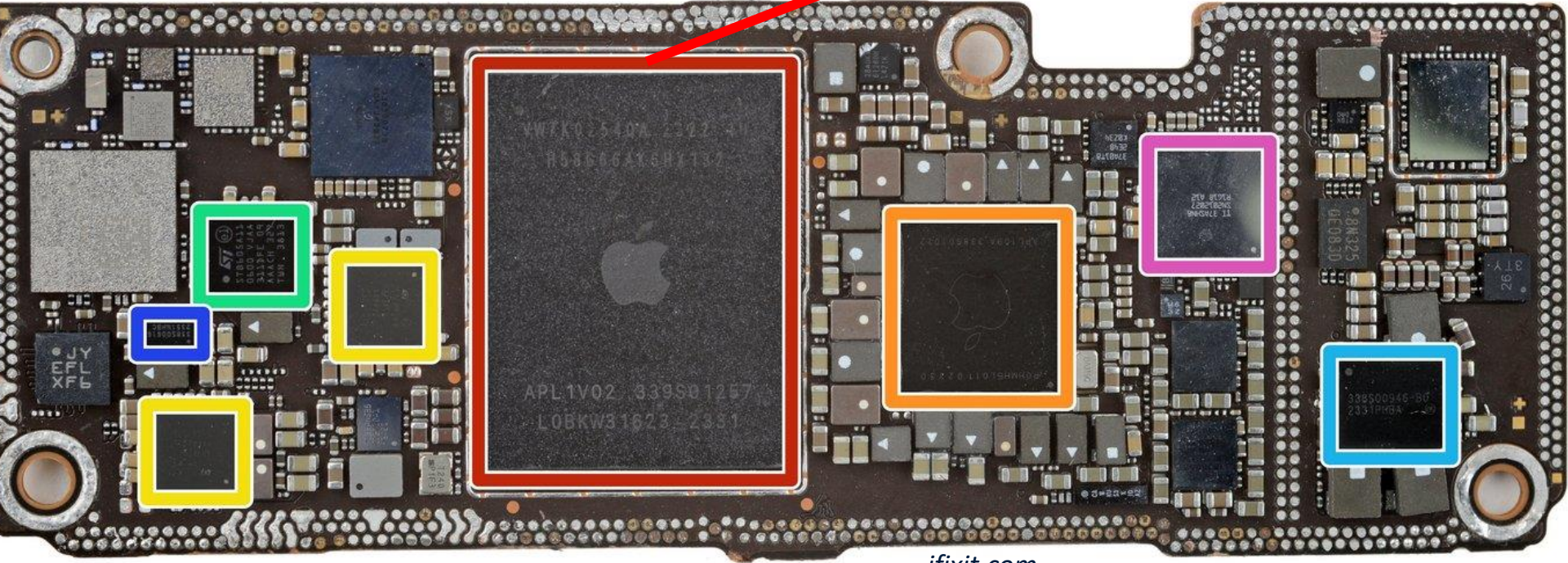


ARM processors are everywhere!



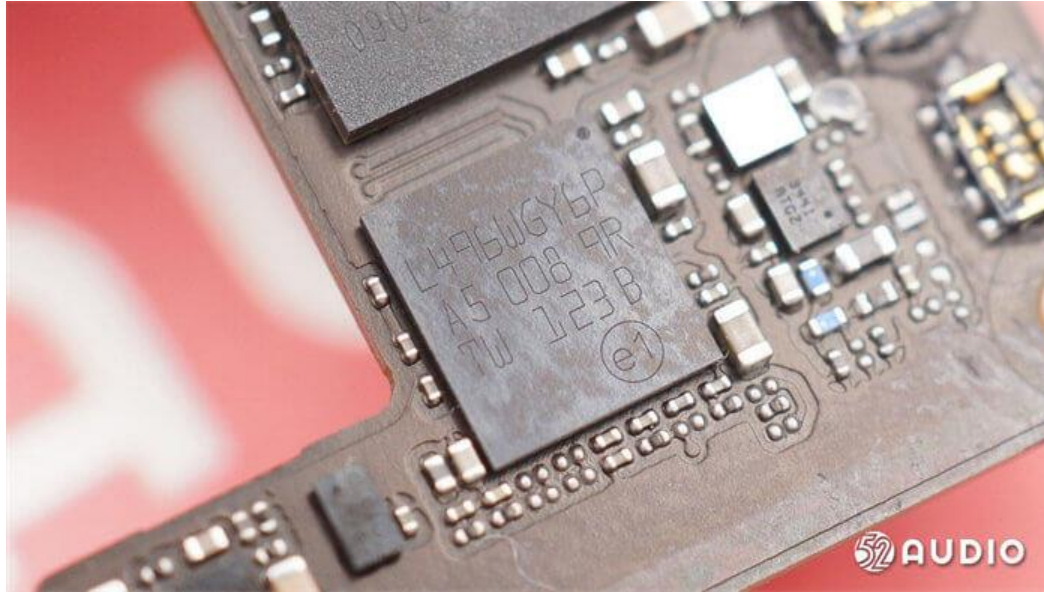
iPhone 15

- A17 Bionic.
- 64-bit system on chip (SoC)
- Arm AArch64 Instruction Set
- Launched in Sept. 2022



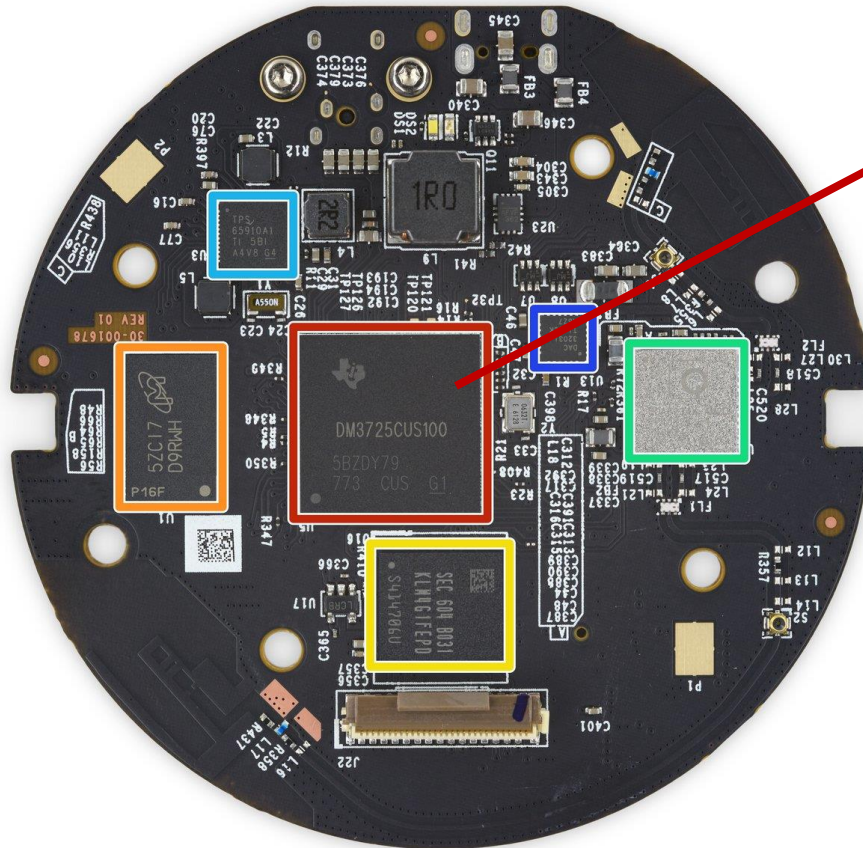
ifixit.com

Apple AirPods 3



- On Charge Station:
STM32L496: Arm Cortex-M4 32-bit MCU+FPU

Amazon Echo (Alexa)



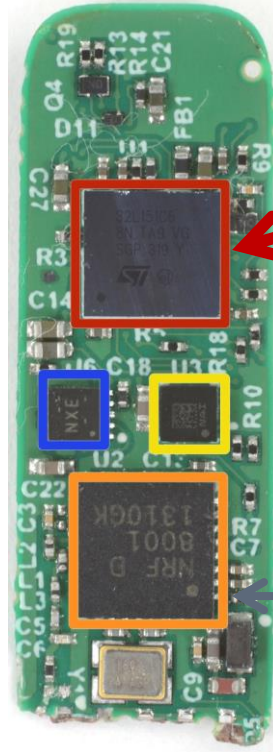
Texas Instruments
DM3725 Digital Media
Processor

- ARM Cortex-A8



<http://www.ifixit.com>

Fitbit Flex



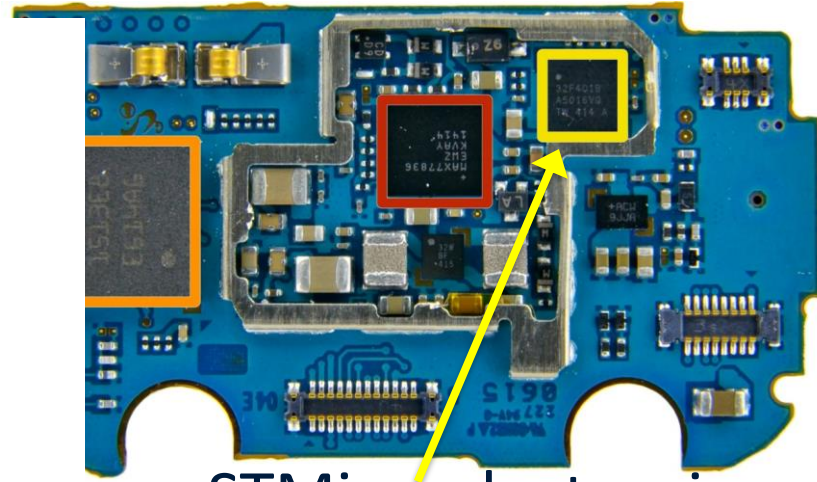
STMicroelectronics **32L151C6**
Ultra Low Power ARM **Cortex**
M3 Microcontroller

Nordic Semiconductor
nRF8001 Bluetooth Low
Energy Connectivity IC

Samsung Galaxy Gear

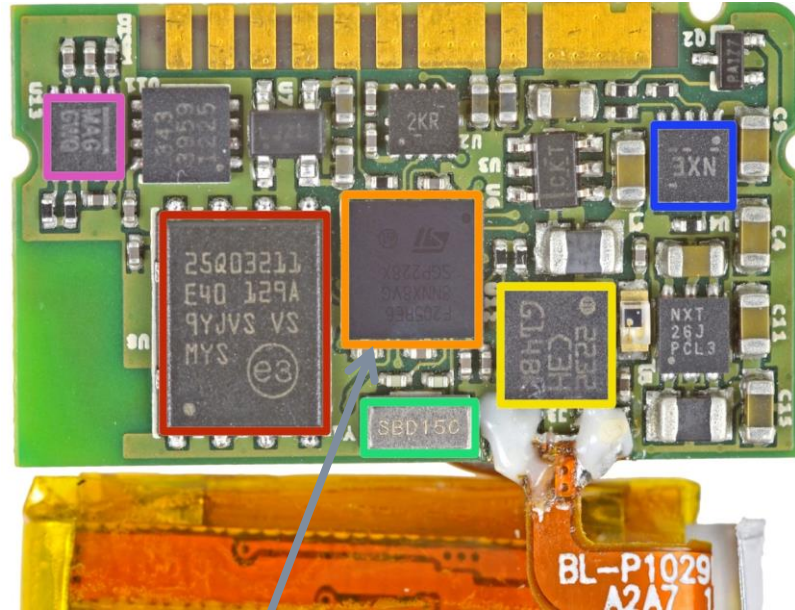


source: ifixit.com



- STMicroelectronics STM32F401B **ARM-Cortex M4** MCU with 128KB Flash

Pebble Smartwatch



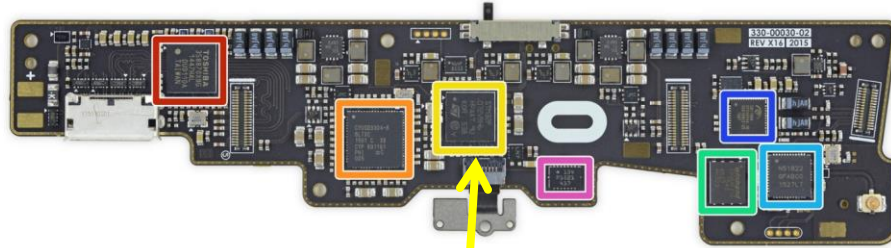
source: ifixit.com

- STMicroelectronics STM32F205RE **ARM Cortex-M3** MCU, with a maximum speed of 120 MHz

Oculus VR

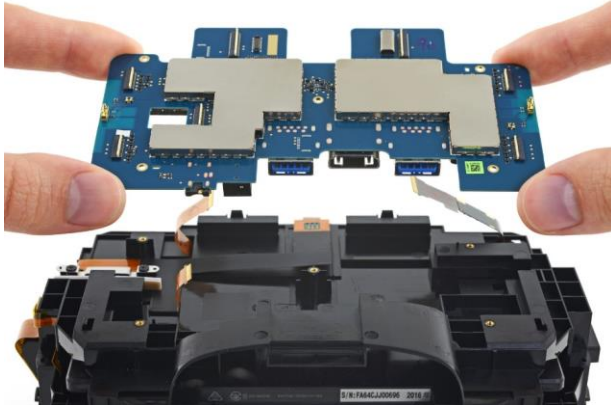


source: ifixit.com

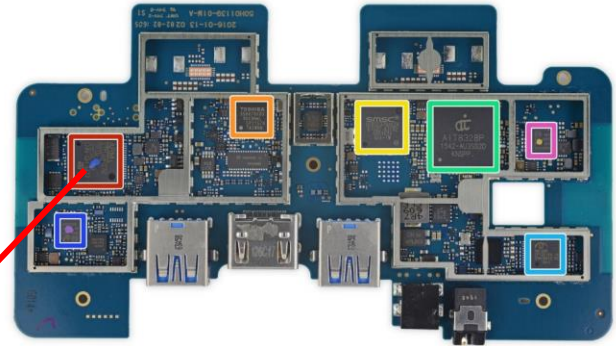


- Facebook's \$2 Billion Acquisition Of Oculus in 2014
- ST Microelectronics STM32F072VB **ARM Cortex-M0** 32-bit RISC Core Microcontroller

HTC Vive

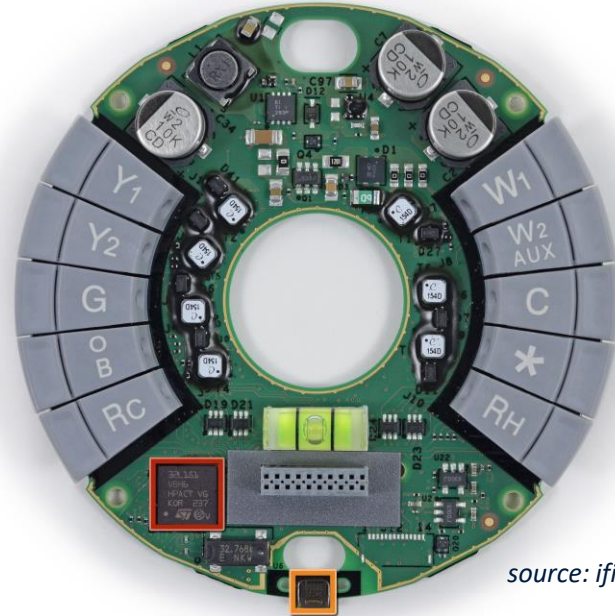


source: ifixit.com



STMicroelectronics 32F072R8 **ARM Cortex-M0**
Microcontroller

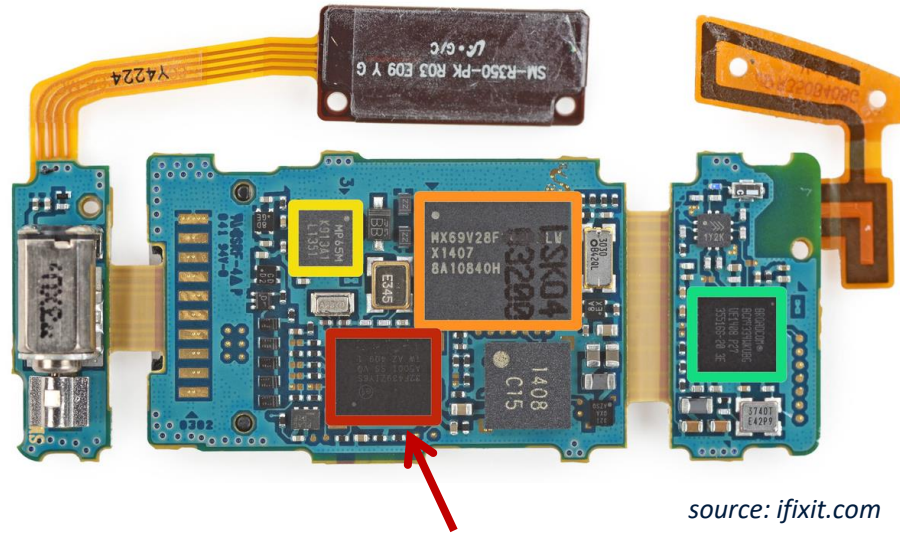
Nest Learning Thermostat



source: ifixit.com

- ST Microelectronics **STM32L151VB** ultra-low-power 32 MHz ARM **Cortex-M3** MCU

Samsung Gear Fit Fitness Tracker



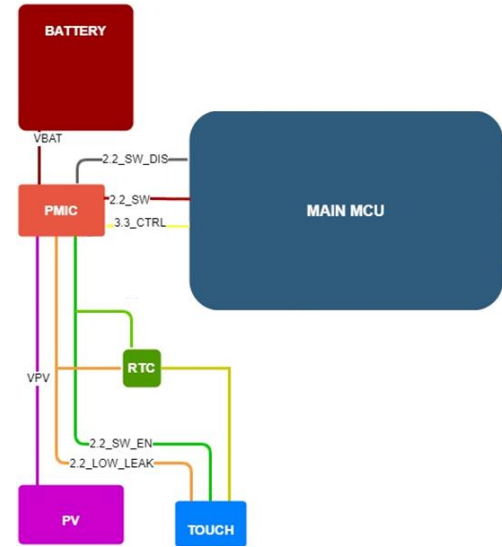
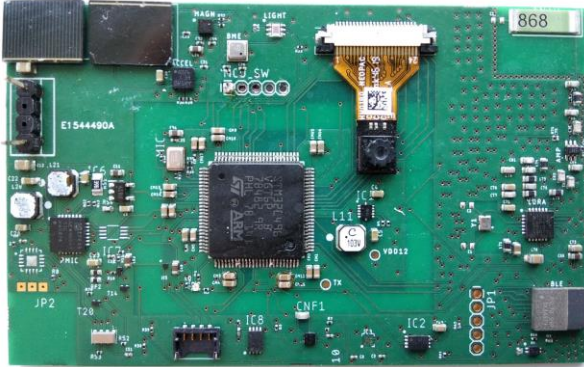
source: ifixit.com

- STMicroelectronics **STM32F439ZI** 180 MHz, 32 bit **ARM Cortex-M4** CPU

Arduino and Raspberry PI



AMANDA



What are the advantages of ARM processors?

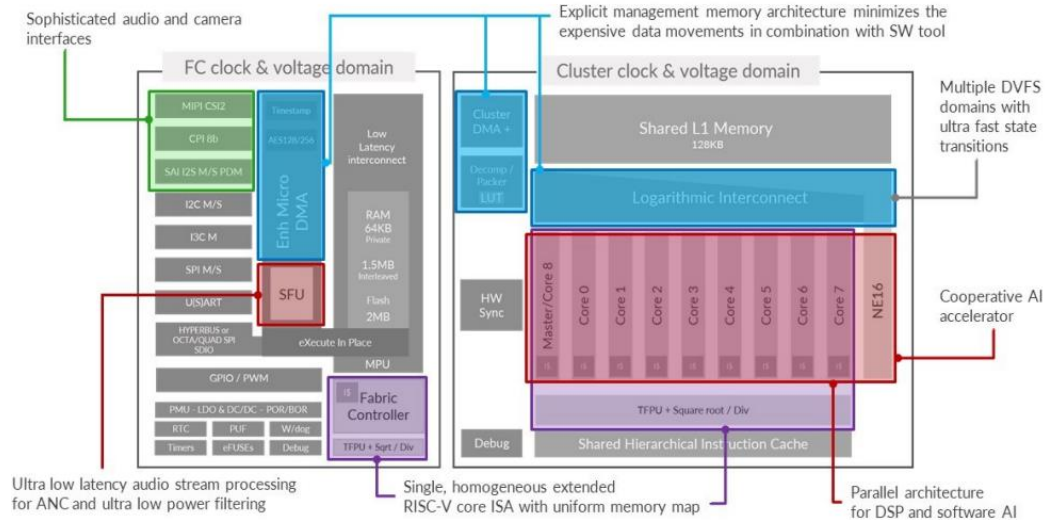
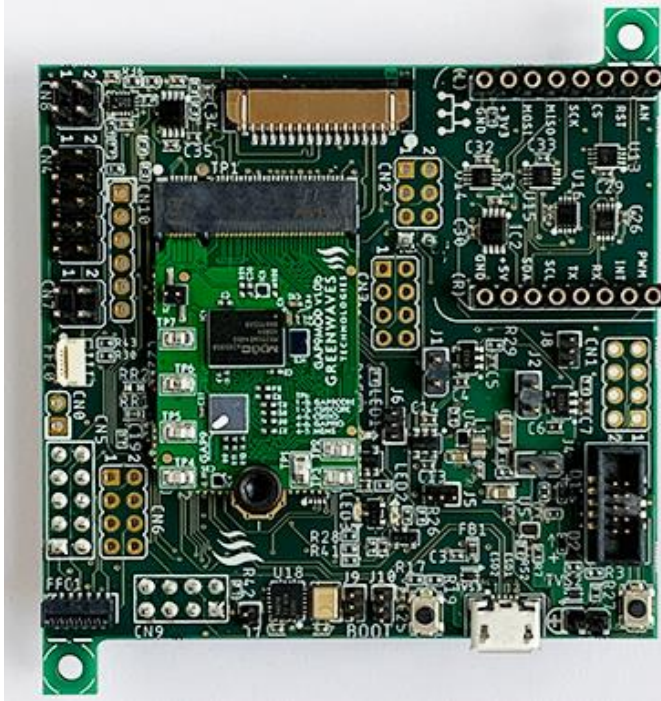
- **Energy Efficiency:**
 - ARM processors are designed for low power consumption
- **Cost-Effectiveness:**
 - Their simple architecture allows for smaller die sizes, reducing manufacturing costs
- **Scalability:**
 - ARM's modular architecture
- **High Performance for specific tasks:**
 - Excellent performance for tasks requiring parallel processing and optimized for RISC (Reduced Instruction Set Computing)

...RISC-V processors are the new trend!

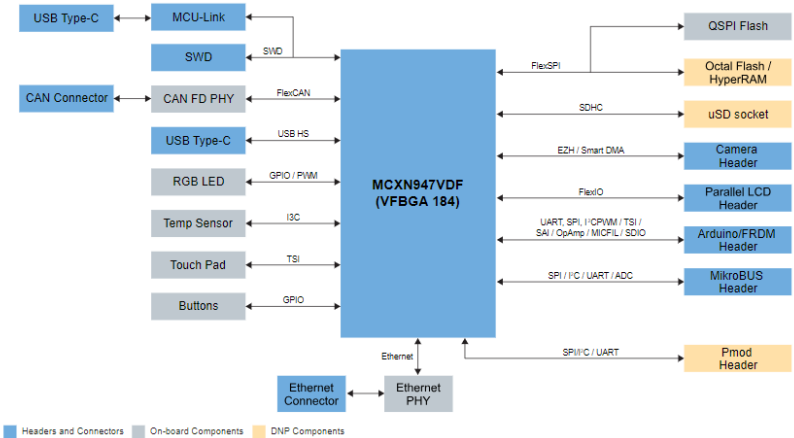
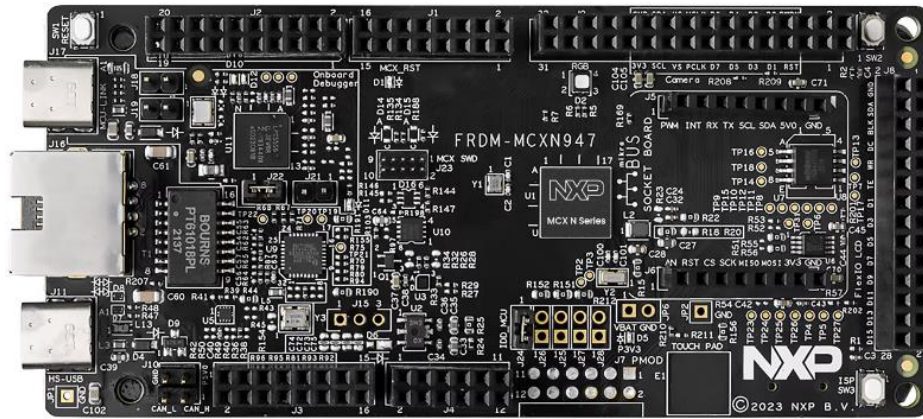
- RISC-V and ARM are both ISAs commonly used in the design of computer processors
- Researchers at Berkeley began the architecture as an **open-source** alternative to ARM to promote greater innovation in technology
- RISC-V has a fixed instruction set architecture with a base integer instruction set
- Includes optional extensions:
 - floating-point arithmetic
 - atomic operations
 - vector processing, etc
- ARM has multiple instruction sets, including ARMv7, ARMv8, and various extensions like NEON for SIMD operations



Some of the SoCs that we play (do research) with in the lab! – GAP9

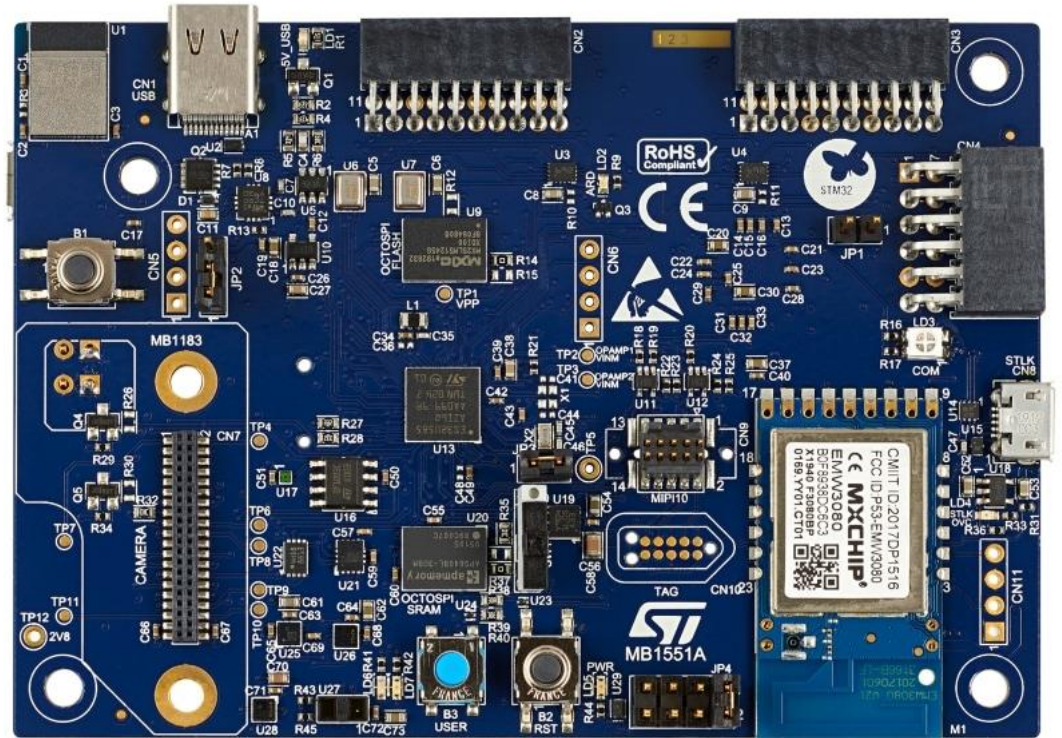


Some of the SoCs that we play (do research) with in the lab! – MCX-N947 FRDM



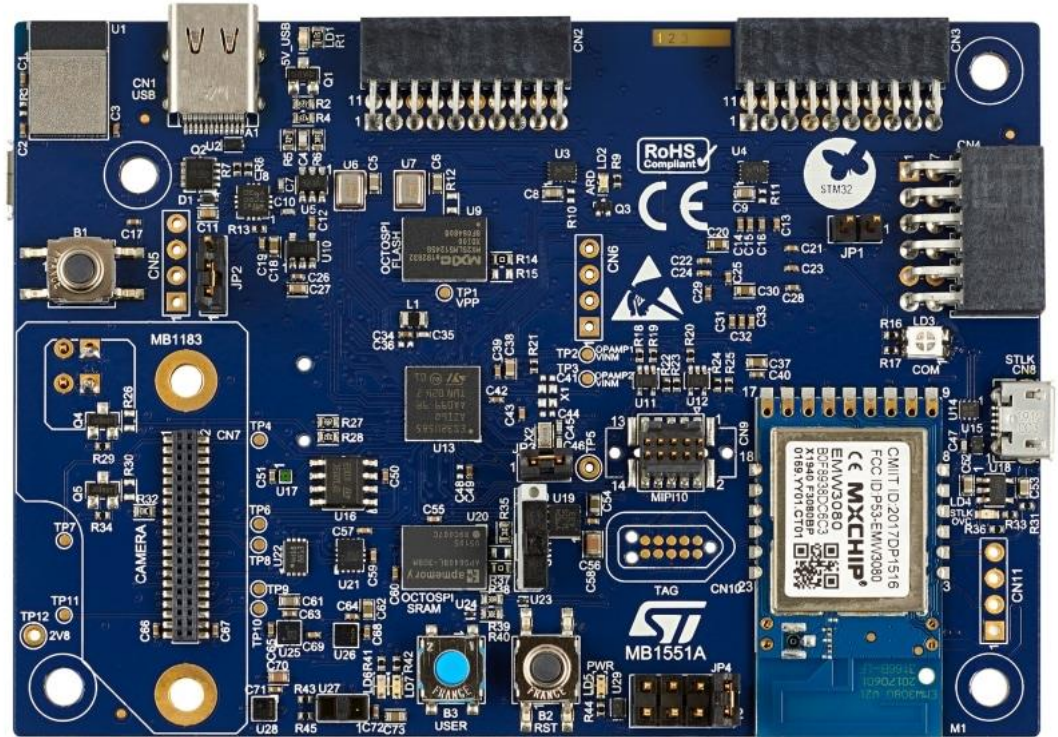
Dual high-performance Arm Cortex-M33 cores running up to 150 MHz, with 2MB of Flash with optional full ECC RAM, a DSP co-processor and an integrated eIQ Neutron NPU.

B-U585I-IOT02A



B-U585I-IOT02A

- **ARM Cortex-M33 core (STM32U585AI)** with UFBGA169 package
- *ARMv8-M architecture with the Main Extension*
- 2Mb Flash memory, 786Kb SRAM
- 512-Mbit Quad-SPI Flash
- 2 digital microphones
- Relative humidity and temperature
- 3-axis magnetometer
- 3D accelerometer and 3D gyroscope
- Pressure sensor, barometer
- Time-of-flight and gesture-detection sensor
- Ambient-light sensor
- 802.11 b/g/n WiFi
- BLE 5.4



Embedded Programming

History of computers

Dr. Charis Kouzinopoulos

Department of Advanced Computing Sciences

Maastricht University

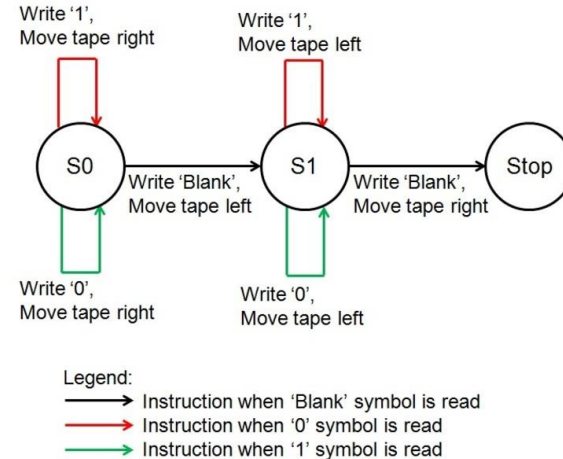
2024-2025

charis.kouzinopoulos@maastrichtuniversity.nl



History of computers

- Alan Turing, a British scientist and mathematician, presents the principle of a universal machine in 1948, later called the Turing machine.
- A mathematical model of computation describing an abstract machine that manipulates symbols on a strip of tape according to a table of rules



History of computers

- ENIAC, the world's first general-purpose electronic computer (World War II)
- Completed in 1945
- Used for computing artillery-firing tables
- Each of the 20 10-digit registers was 2 feet long. In total, ENIAC used 18,000 vacuum tubes.
- The machine is the first "automatic, general-purpose, electronic, decimal, digital computer"



History of computers

- UNIVAC I, the first commercial computer for business and government applications in the United States (1951)



History of computers

- Cray-1, the first commercial vector supercomputer, announced in 1976
- The fastest computer for scientific applications and the computer with the best price/performance for those applications



History of computers

- VAX-11, a family of 32-bit superminicomputers, running the Virtual Address eXtension (VAX) ISA
- Developed and manufactured by the Digital Equipment Corporation (DEC) in 1976
- Powerful machines that also offered the ability to run user mode PDP-11 code (a series of 16-bit minicomputers)
- VAX-11/780, code-named "Star", was introduced on 1977
- The KA780 central processing unit (CPU) is built from Schottky transistor-transistor logic (TTL) devices and has a 200ns cycle time (5 MHz) and a 2 KB cache



History of computers

- Steve Jobs and Steve Wozniak co-found Apple Computer on April Fool's Day in 1976.
- They unveil Apple I, the first computer with a single-circuit board and ROM



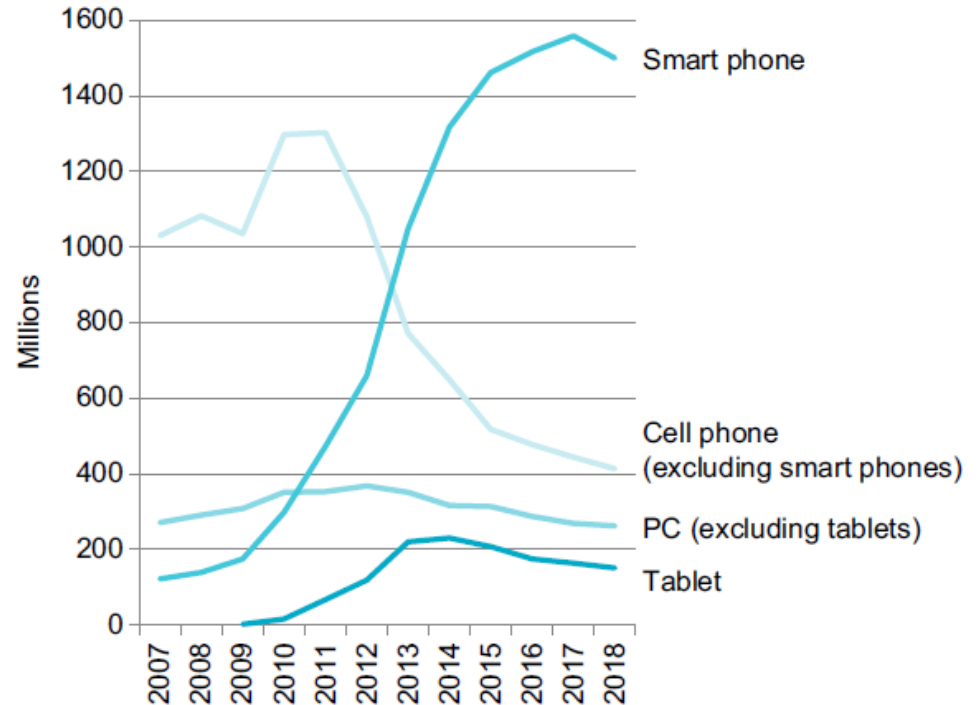
History of computers

- "Acorn," IBM's first personal computer, is released onto the market in 1981 at a price point of \$1,565
- Acorn used the MS-DOS operating system from Windows
- Optional features include a display, printer, two diskette drives, extra memory, a game adapter and more



The post-pc era: smartphones and IoT

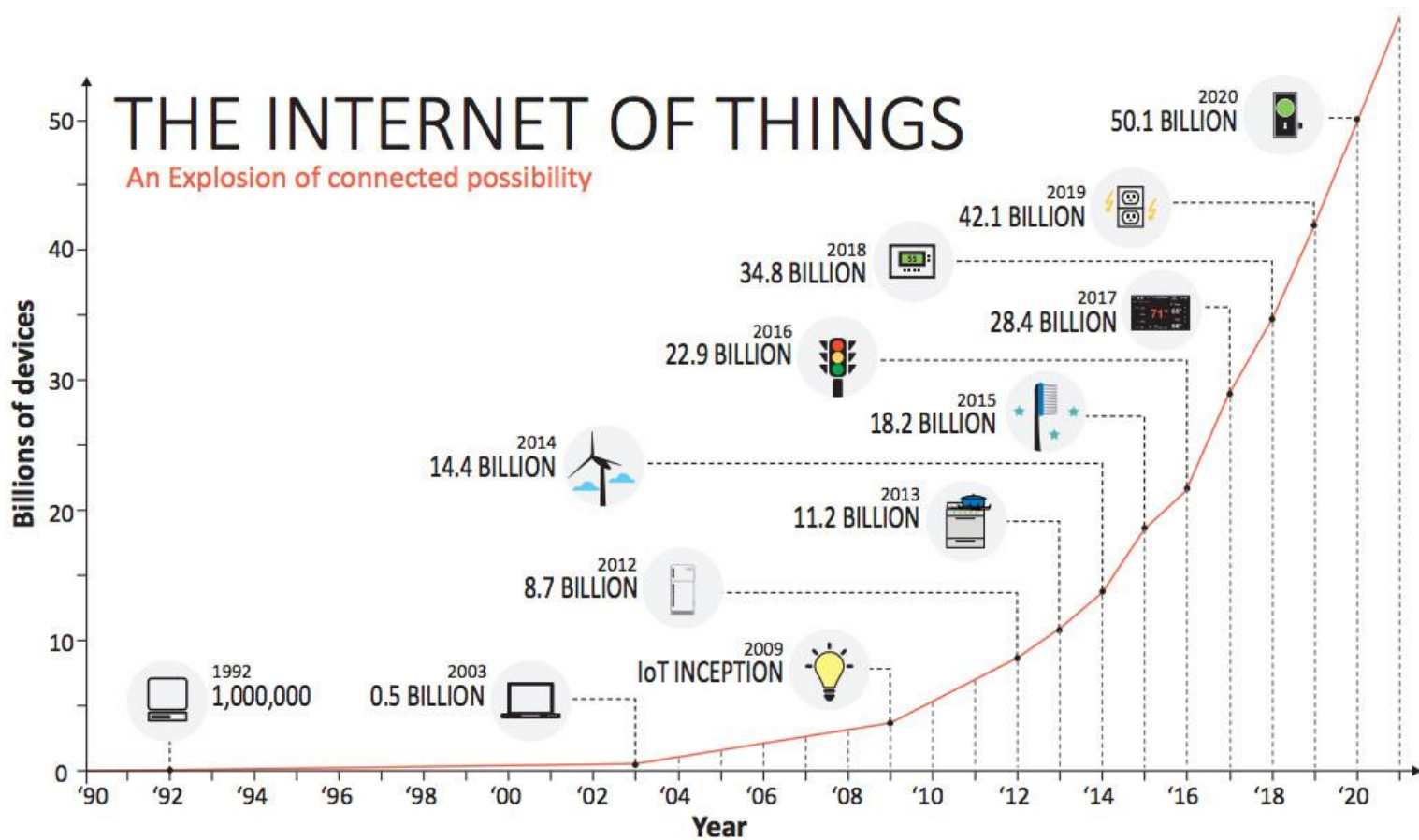
- Smart phones represent the recent growth in the cell phone industry, and they passed PCs in 2011





Overview of IoT

Definition and origins



The language of the computers

- To speak directly to electronic hardware, you need to send electrical signals. The easiest signals for computers to understand are on and off, and so the computer alphabet is just two letters: 0 and 1
- We refer to each “letter” as a **binary digit** or **bit**
- Computers respond to our commands, which are called **instructions**. Instructions are just collections of bits that the computer understands. For example, the bits:

1001010100101110

tell one computer to add two numbers

- Using numbers for **both** instructions and data is a **foundation of computing**

The language of the computers

- The first programmers communicated to computers in **binary numbers**, but this was so tedious that they invented new notations that were closer to the way humans think
- These notations were manually translated to binary, but this process was tiresome. The pioneers invented the **assembler** to translate from symbolic notation to binary
- The assembler translates a symbolic version of an instruction into the binary version. For example, the programmer would write:

add r3, r2



Assembly language

- and the assembler would translate this notation into:

1001010100101110



Machine language

The language of the computers

```
unsigned int var1 = 3;  
unsigned int var2;  
var2 = var1 + 5;  
var2++;
```



08000284: 05 33

adds r3, #5



15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0 0 1			op		reg			imm8							

001 10 011 00000101
op reg imm8



This table shows the decode field values and the associated instructions:

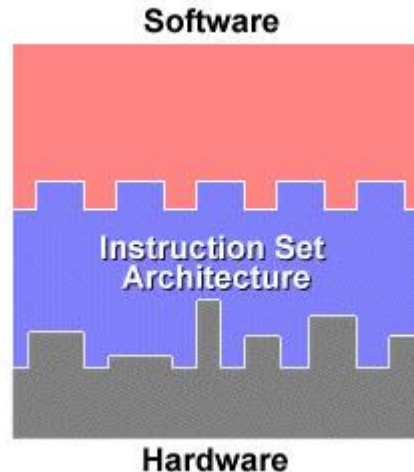
op	Instruction
00	MOV (immediate) T1
01	CMP (immediate) T1
10	ADD (immediate) T2
11	SUB (immediate) T2

51 dec (33 hex)

ARMv8M architecture reference **C2.2.1.3**

The language of the computers

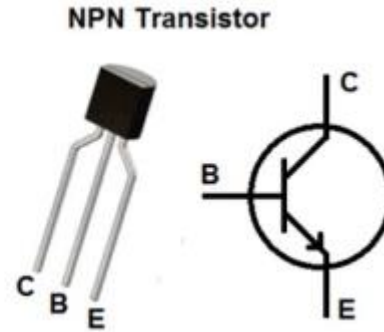
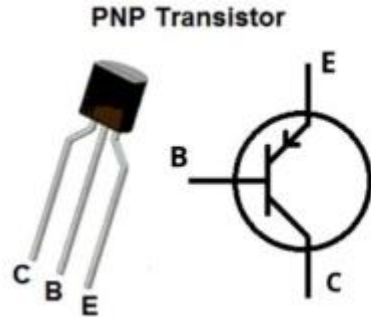
- The words of the vocabulary are called **instructions**, and the vocabulary itself is called the **instruction set architecture (ISA)**, or simply **architecture**, of a computer
- The ISA acts as an interface between the hardware and the software, specifying both **what** the processor is capable of doing as well as **how** it gets done



Meet the transistor

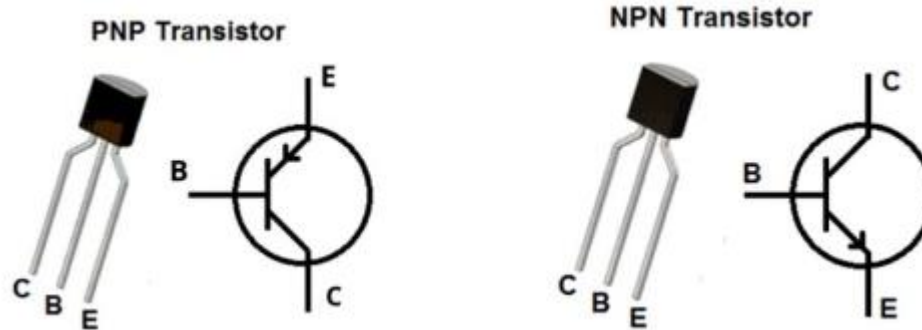
Why do the computers speak in binary? Meet the transistor:

- ***A transistor is a semiconductor device used to amplify or switch electrical signals and power***
- ***It is simply an on/off switch controlled by electricity!***
- ***One of the basic building blocks of modern electronics***



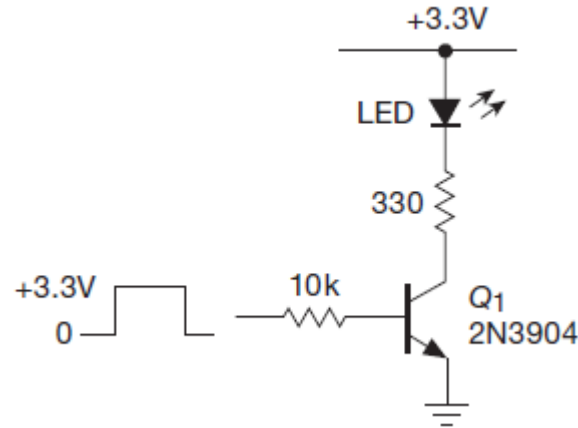
Meet the transistor

- The highest transistor count in a consumer microprocessor as of June 2023 is 134 **billion** transistors, in Apple's ARM-based dual-die M2 Ultra SoC, fabricated using TSMC's 5 nm semiconductor manufacturing process



Meet the transistor

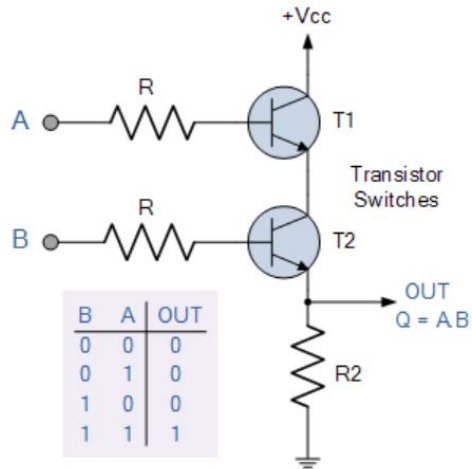
A classic circuit:



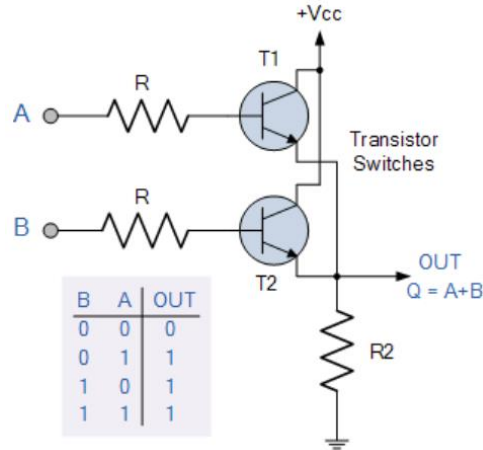
Driving an LED from a “logic-level” input signal, using an npn saturated switch and series current-limiting resistor

Meet the transistor

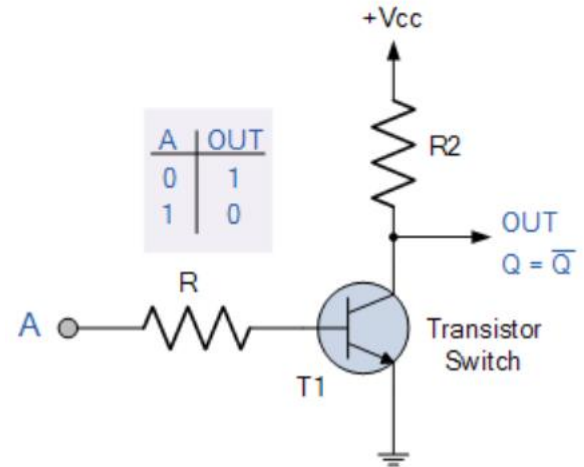
- Logic gates can be constructed out of transistors:



2-input Transistor **AND** Gate



OR Gate



NOT Gate

High-level programming languages

- Programmers today owe their to the creation of high-level programming languages and **compilers** that translate programs in such languages into **instructions**

```
swap(size_t v[], size_t k)
{
    size_t temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```

↓

Compiler

↓

```
swap:
    slli x6, x11, 3
    add x6, x10, x6
    lw x5, 0(x6)
    lw x7, 4(x6)
    sw x7, 0(x6)
    sw x5, 4(x6)
    jalr x0, 0(x1)
```

↓

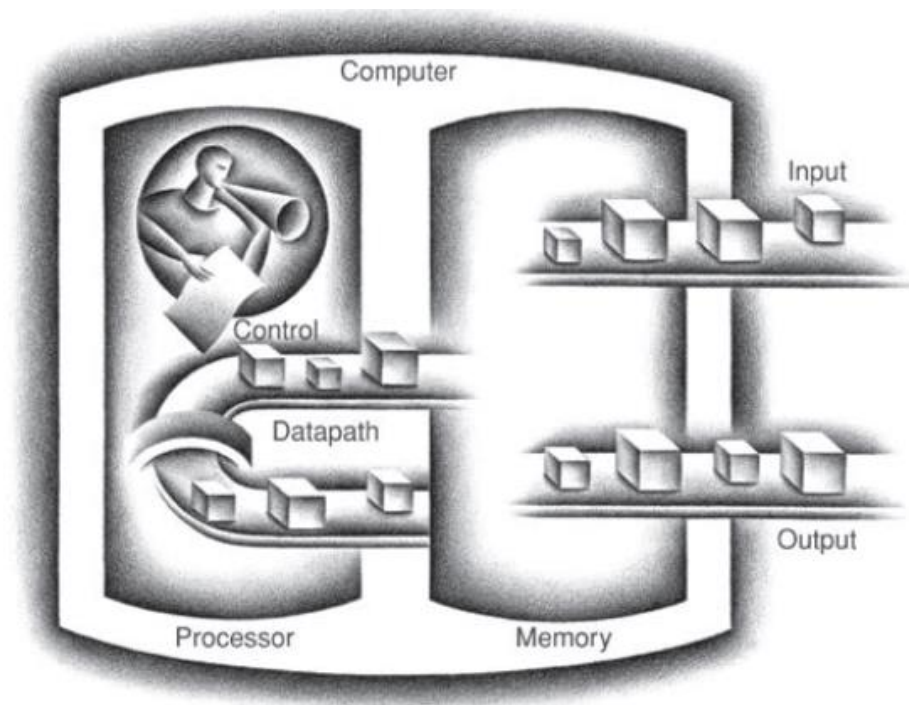
Assembler

↓

```
00000000001101011001001100010011
00000000011001010000001100110011
00000000000000110011001010000011
00000000100000110011001110000011
00000000011100110011000000100011
00000000010100110011010000100111
0000000000000000100000001100111
```

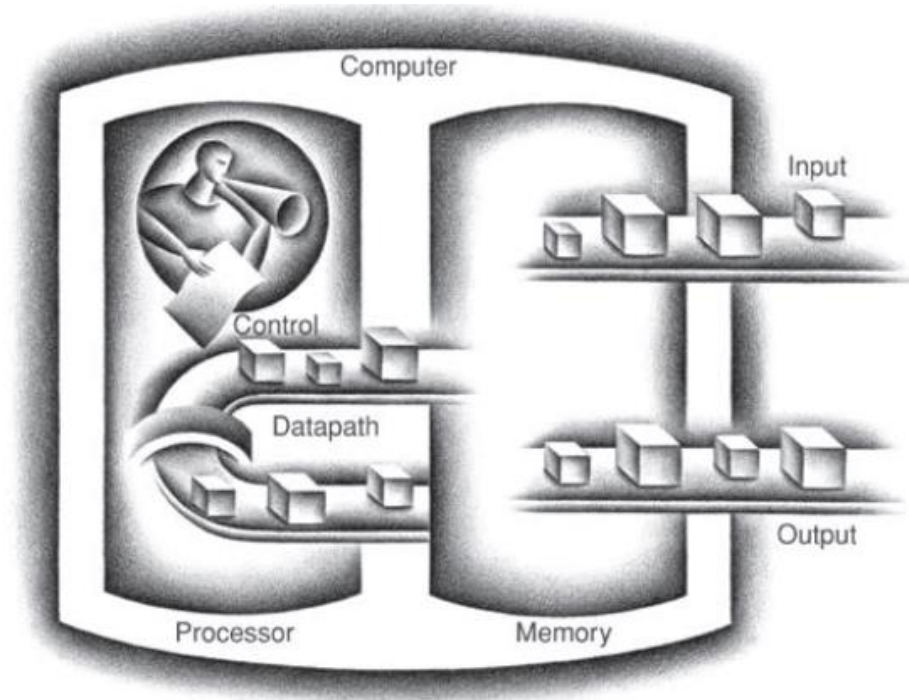
Computer organization

- The five classic components of a computer are input, output, memory, datapath, and control, with the last two sometimes combined and called the processor.
- This organization is **independent** of hardware technology: you can place every piece of every computer, past and present, into one of these five categories



Computer organization

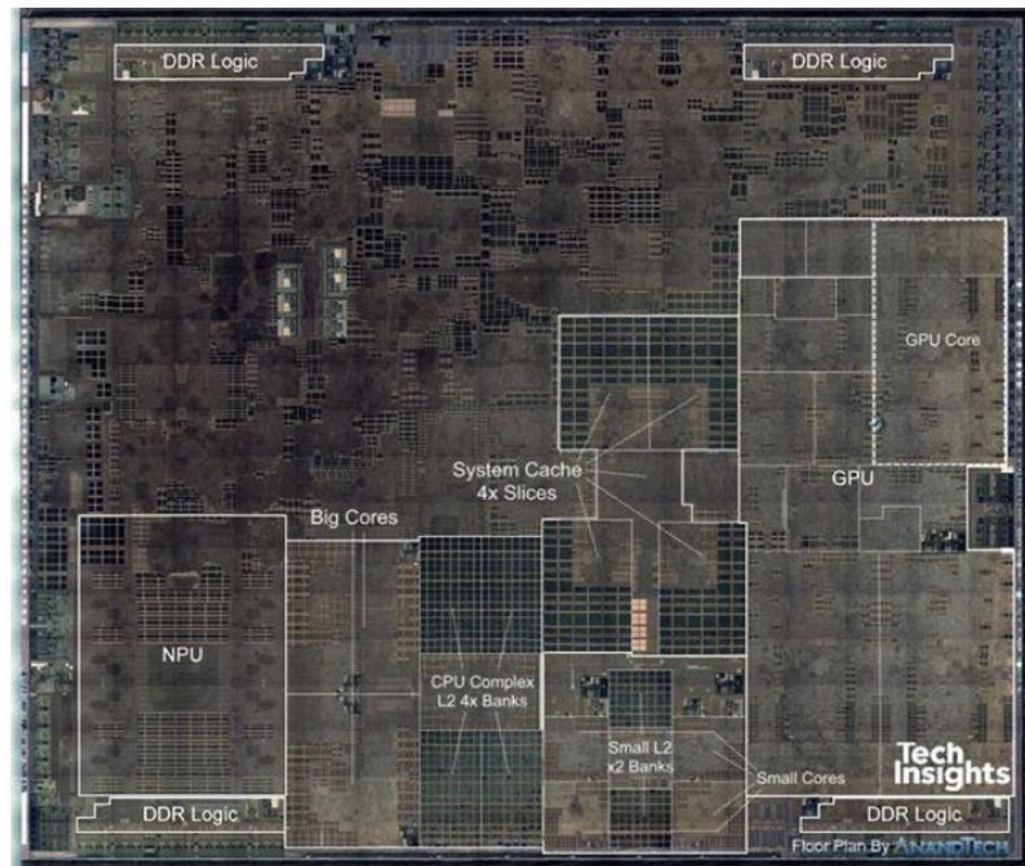
- The processor gets instructions and data **from memory**
- Input writes data **to memory**, and output reads data **from memory**
- Control sends the signals that determine the operations of the datapath, memory, input, and output.



Computer organization

Apple A12 SoC: The processor integrated circuit inside the A12 package. 8.4 x 9.91 mm chip size.

- Has two identical ARM processor cores (middle), four small cores (lower right)
- Has a GPU (right) and an NN accelerator (left)
- There are L2 cache memory banks for the big and small cores (middle)
- At the top and bottom are interfaces to the main memory (DDR DRAM)



Embedded Programming

The power wall

Dr. Charis Kouzinopoulos

Department of Advanced Computing Sciences

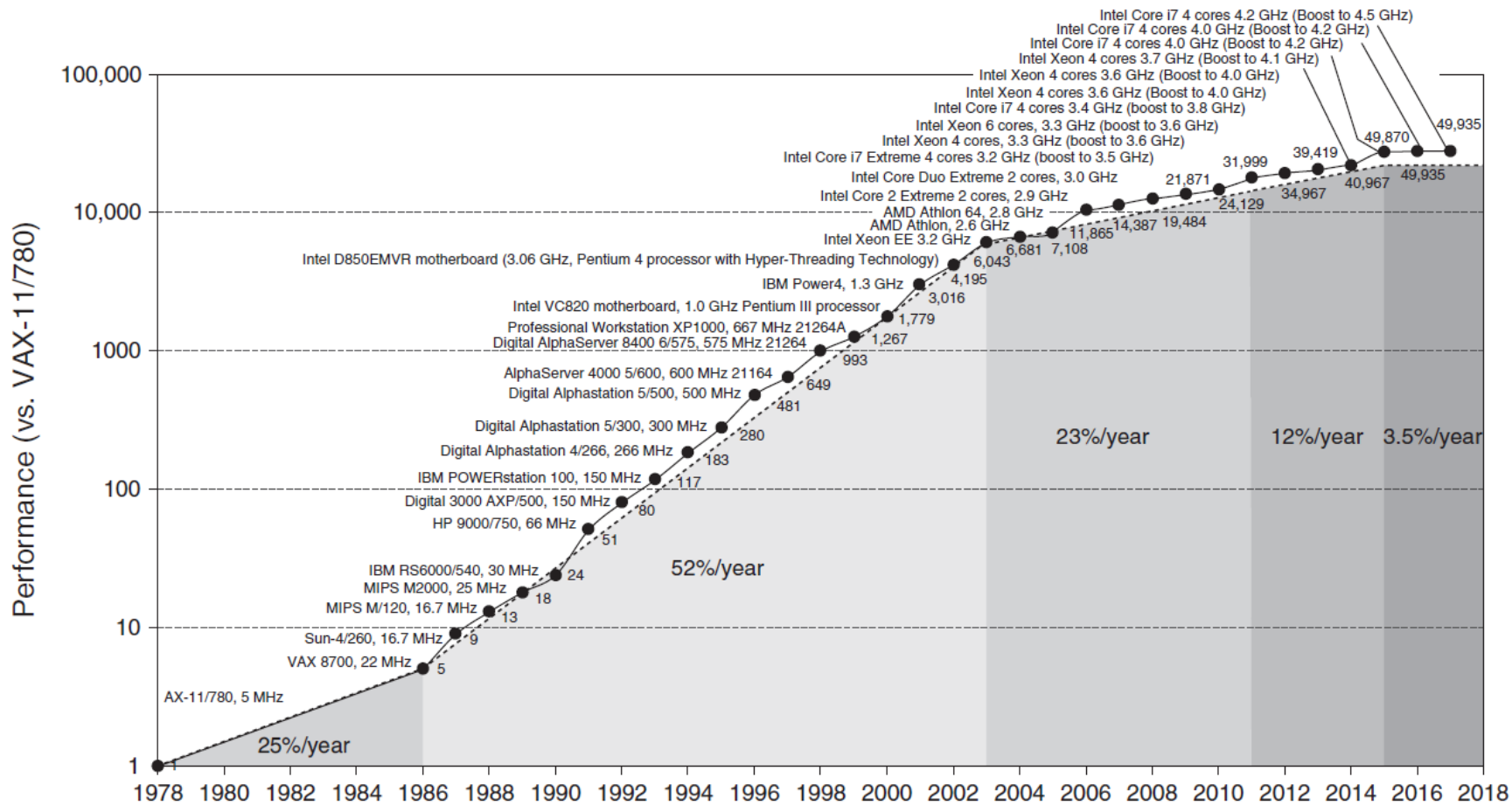
Maastricht University

2024-2025

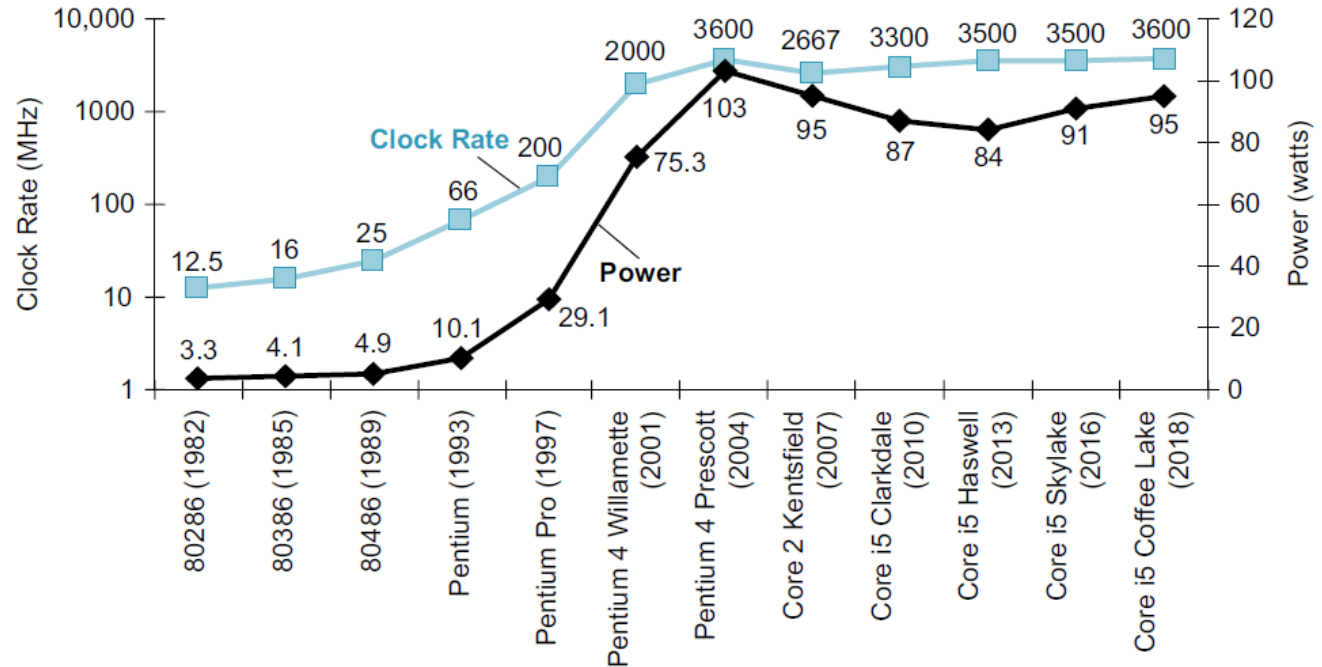
charis.kouzinopoulos@maastrichtuniversity.nl



Maastricht University



The power wall



increase in clock rate (blue) and power (black) of nine generations of Intel CPUs

Takeaways

- Learning objectives and outline of the course
- What is an embedded system and why are they important – and everywhere?
- ARM vs RISC-V
- History of computers
- What language do the computers speak?
- What is a transistor and why is it the most essential computer component?
- Computer organization and the power wall